ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Разработка системы чат-бота поддержки beauty-coworking

Актуальность: поддержка клиентов требует единых правил ответов и прозрачного учета обращений

Технологическая база: контейнеризация и модульная схема сервисов

Предмет защиты

Демонстрационный проект информационной системы поддержки

Детерминированный поток диалога и маршрутизация заявок

Демонстрация сценариев: FAQ, бронирование, передача оператору

2026-05-12

Объект, предмет и цель исследования

Ключевые формулировки для академической защиты

Объект

Поддержка клиентов сервисного бизнеса на базе чат-интерфейса и операторского контура

Предмет

Проектирование и реализация ИС

распределяющей реплики по намерению

и обеспечивающей эскалацию оператору

Цель

Разработать чат-систему, которая получает запрос клиента, определяет сценарий,

выдает корректный ответ или направляет на следующий этап обработки человеком.

Задачи: описание домена beauty-coworking, реализация маршрутизации диалогов,

создание operator queue и операторского контура, верификация локальными gate-командами.

Роли, доступ и безопасность

Клиент: начинает сессию с веб-чата, получает меню и ответы по доступным сценариям.

Система: определяет intent, выбирает ветку сценария и проверяет необходимость операторской помощи.

Оператор: просматривает очередь обращений, принимает и завершает диалог, возвращает итог пользователю.

Новые обращения создаются через отдельный контрактный слой, а не через прямой доступ к внутренним таблицам.

Проект выполняется как demo-реализация с фиктивными внешними провайдерами и контролируемыми данными.

Ограничения

Нет заявок на реальную авторизацию сотрудников в внешних ID-системах.

Не используется продовая связь с внешними платежными и бронированием API.

Функционал информационной системы

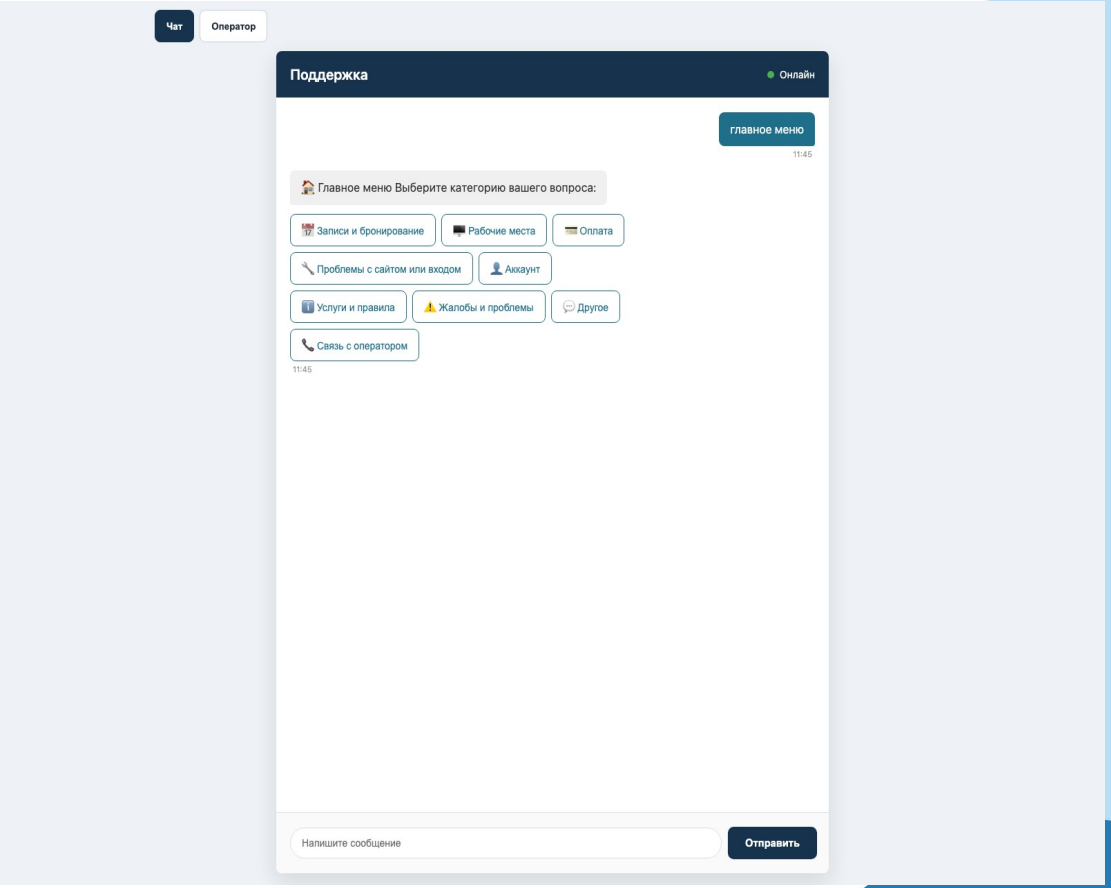
Единое главное меню с быстрым выбором тематических блоков.

FAQ по услугам, правилам и рабочему графику.

Поиск заявки по идентификатору и вывод статуса.

Навигация по тарифам, локациям и рабочим зонам.

Автоматический перевод на оператора при нестандартном запросе.



Снимки экрана

02-main-menu.png демонстрирует стартовую навигацию и категорийный выбор.

Архитектура решения

website
:8081

decision-engine
:8080

nlp-service
:8082

postgres+pgvector
:5442

mock providers
:8090

Фронт и API идут в web-канал, который передает сообщение в decision-engine.

Decision-engine использует локальные сценарии, таблицы и NLP-каталог для маршрутизации.

pgvector и fixtures обеспечивают воспроизводимые решения без live-зависимостей.

Операторская очередь и статусы обращений доступны из backend для ручного завершения.

Модель данных

Сущности в ядре: сессии, сообщения, intents, действия, статусы

Для диалогового хранения также используются структуры очереди и операторских задач

Справочники бизнес-направлений связаны с FAQ и семантическими примерами

Данные о бронированиях и платежах хранятся как структурированный вход для deterministic flow

Для всех сущностей задаются простые контракты обмена и минимальная схема миграций.

Данные проверки

fake-hash embedding v1, размерность 384

seed data: 40 intents, 437 примеров

knowledge chunks 25, demo-bookings 4

Схема потоков

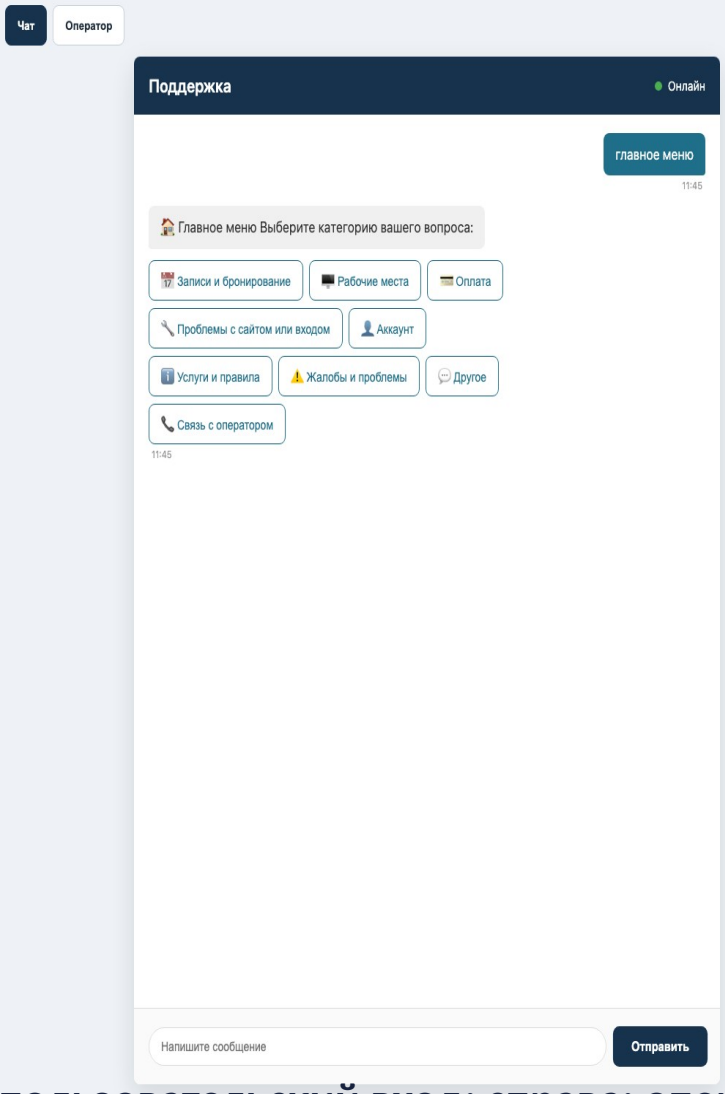
Пользовательское сообщение

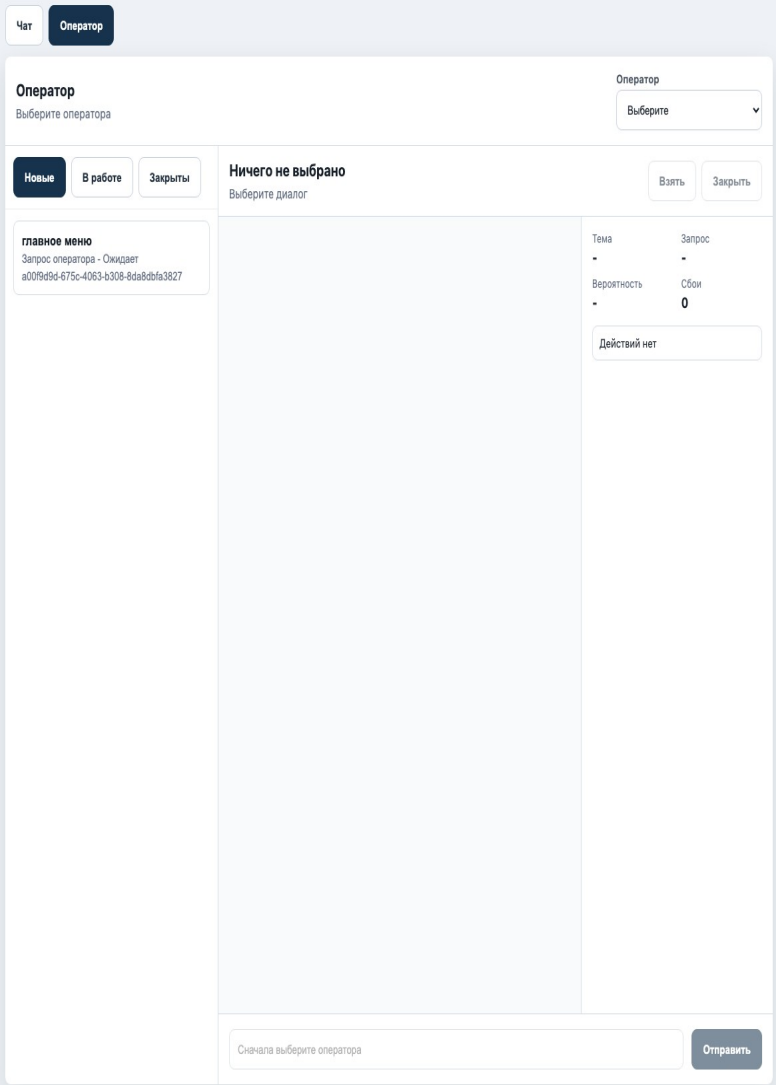
→ session + intent

→ catalog / action

→ response or operator task

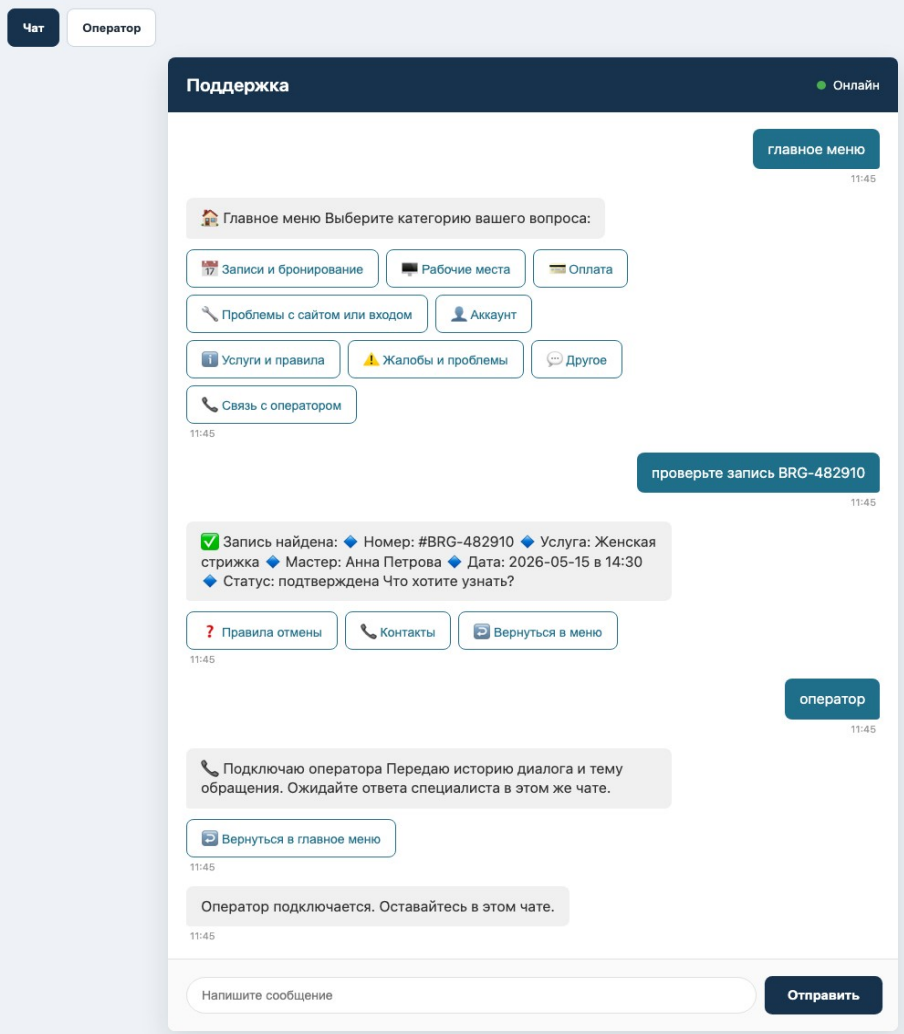
Интерфейс ключевых экранов





Слева: пользовательский вход; справа: операторский интерфейс и очередь.

Сценарий обращения и передача оператору



1) Пользователь отправляет запрос в чат.

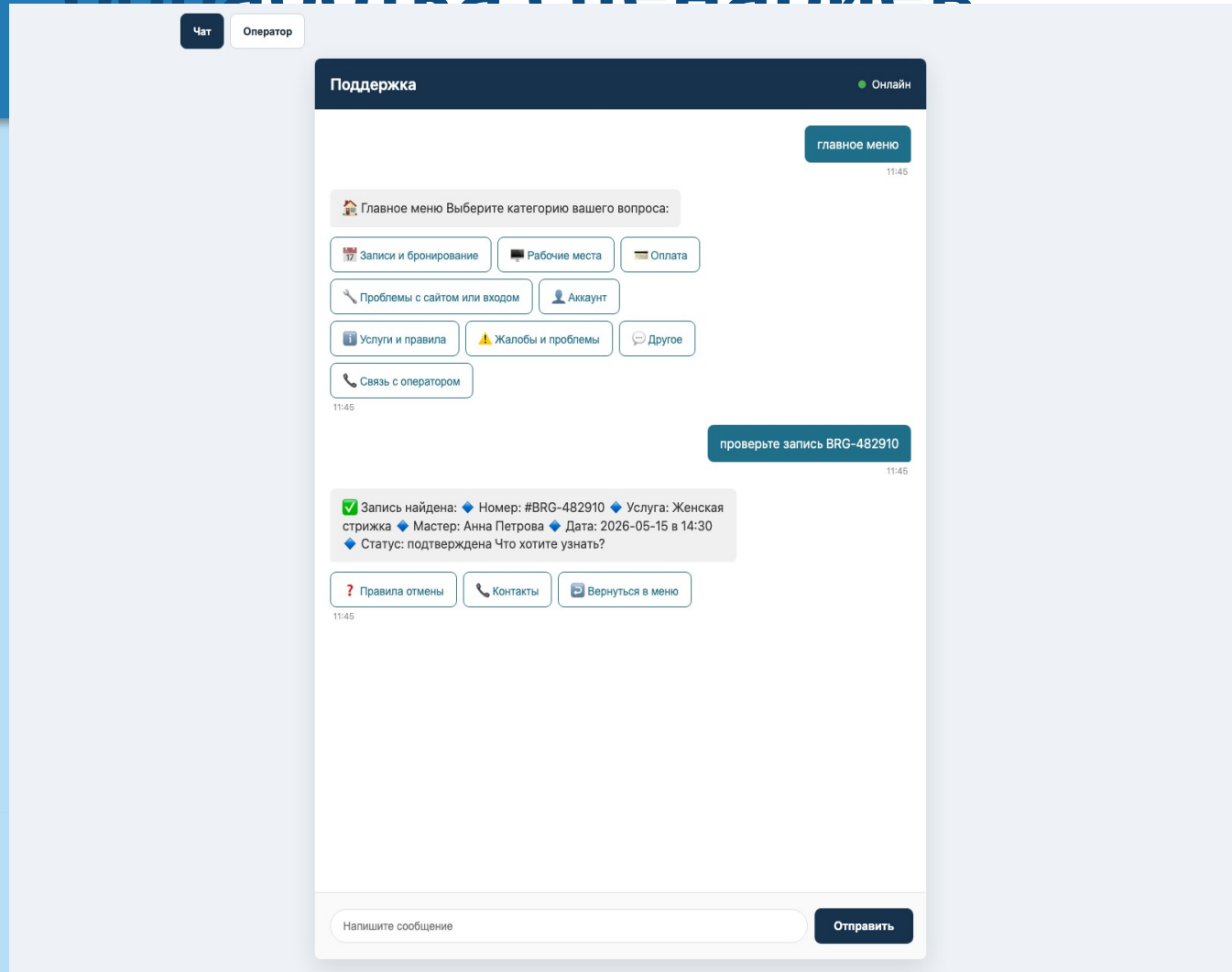
2) Decision-engine определяет intent и проверяет, есть ли автоматический ответ.

3) При сложном случае формируется задача и заявка попадает в очередь.

4) Оператор принимает задачу, завершает диалог и возвращает результат пользователю.

5) Все шаги фиксируются в операционных сущностях.

Поиск записи, операторская очередь и обработка сценариев



Пример сценария: поиск брони по идентификатору и выдача найденных параметров.

Результат передается в структуру разговора без ручного форматирования.

Если контактная информация и статус не распознаны — диалог переводится на оператора.

Подтверждены ограничения: reproducible демонстрационный runtime и mock-провайдеры.

Проверка работоспособности и результаты

Практика валидации: make semantic-gate; make check-core; scripts/smoke-compose.sh;

Критические e2e проверки: make e2e-smoke; make e2e-full.

Информационная система локально воспроизводима и проверена на наборе фиктивных сценариев.

Контракты с внешним миром ограничены mock-слоем: это демонстрационный, детерминированный проект.

Общий итог: покрытие функционала, архитектурной трассируемости и операционной очереди подтверждено.

Фактография

pgvector: 0.8.2

fake-hash embedding v1

seed data: 4 demo accounts, 4 demo bookings, 5 demo payments

Спасибо за внимание